

JavaScript im Einsatz

REST

RESTful Services

REpresentational State Transfer

- »Paradigma« zur Architektur verteilter Systeme, die den Prinzipien webbasierter Systeme folgt
- Abstraktion des Verhaltens des »World Wide Web«
- REST ist ausschließlich eine Architektur
 - > Web Services (SOAP-basiert) sind beides: Standard und Architektur
- Die Architektur beschreibt die Interaktion von Anwendungen gemäß dem webbasierten Interaktionsmodell
 - > Verwendung von HTTP als Transportprotokoll
 - > Feststehende Operationen
 - > Interaktion mit „Ressourcen“ als Dreh- und Angelpunkt

REST ist eine ressourcenorientierte Architektur (ROA)

RESTful Services

Konzept

URL

`http://localhost/StudentAPI/studenten/4002458`

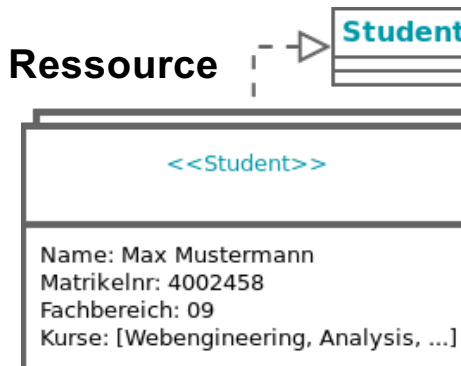
Repräsentation (JSON)

```
{  
  "Name": "Max Mustermann",  
  "Matrikelnr": 4002458,  
  "Fachbereich": "09",  
  "Kurse": ["Webengineering",  
            "Analysis", ...]  
}
```

identifiziert

repräsentiert

Ressource



- Ressourcen (HTML-Seiten, Bilder, Objekte) werden über URLs angefordert oder verändert
- Übertragen der Objekte in verschiedenen Repräsentationen möglich, z.B.
GET StudentAPI/studenten/4002458
als JSON-Objekt (andere Rückgabeformate möglich)
- Mit dem Anlegen/Ändern einer Ressource beim Server können dortige Prozesse angestoßen werden

RESTful Services

Eigenschaften

Adressierbarkeit

- Jede Ressource ist über eine URL identifizierbar

Zustandslosigkeit

- Keine Anwendungszustände im Server (nur Ressourcenzustände)

Einheitliche Schnittstelle

- CRUD-Operationen (GET, POST, PUT, DELETE)

Entkopplung von Ressourcen und Repräsentation

- Anforderung einer Ressource im gewünschten Format (JSON, XML, HTML, ...)

Hypermedia

- Verwendung von Hyperlinks zur Navigation

- Mit dem Ausliefern einer Repräsentation einer Ressource geht auch die **Verantwortung über die Ressource an die Benutzer über**
- Der Server behält die Ressource zwar weiter vor und kann theoretisch auch diese über die Zeit verändern (durch angestossene Prozesse), jedoch kann jeder Berechtigte eine Operation auf dieser durchführen und somit einen neuen Zustand erzeugen
- Der Server kennt weder alle Details zu zukünftig möglichen Operationen noch pflegt er eine genaue Historie, um so auf einen Kontext zu schließen. Der Server behandelt somit **jeden Aufruf als in sich abgeschlossenen Auftrag ohne impliziten Bezug auf vorhergehende Operationen**.
 - > Keine direkte Unterstützung von Transaktionen über Aufrufgrenzen hinweg
 - > Die Anwendung kann allerdings Attribute schaffen und Hypermedia verwenden, um Operationen zu vernetzen

RESTful Services

Idempotente Operation

- REST verfolgt die Idee, dass jede Operation auf einer Ressource in sich abgeschlossen ist und nicht von eventuell vorher durchgeführten Operationen abhängt (**independent** and **self-contained**)
- Dieses Konzept passt gut zu **idempotenten Operationen**:
 - > $Op(x) = Op(Op(x))$
 - > `i=42` ist idempotent, `i++` nicht
- Das **Anlegen** einer neuen Ressource ist **normalerweise nicht idempotent**, denn eine Create erzeugt jedes Mal eine Ressource. Allerdings könnten im Create ein Unique Identifier genutzt werden, mit dem eine Ressource differenzierbar wird.
- Das Update einer Ressource (PUT) sollte alle Attribute einer Ressource liefern. Hinweis: Ein PATCH (Diff-Notation von JSON-Patch) ist aber nicht immer idempotent.

Zustandslosigkeit und idempotente Operationen passen gut zusammen (At-least-Once-Semantik)

Hinweis: Ggf. kann man über Versionierung und Unique Identifier effizienter Szenarien abbilden

RESTful Services

Hypermedia

- Letztlich wollen wir nicht nur mit einzelnen Ressourcen arbeiten, wir wollen vernetzte Ressourcen und Prozesse abbilden
- Hypermedia liefert die Verlinkung von Ressourcen und kann so als wichtige Stütze für die Implementierung von Interaktionsprozessen/Workflows dienen
- Ressourcen „pflegen“ also die Verlinkung mit anderen Ressourcen

Faktisch stützt die Verlinkung der Ressourcen den Aufbau von Arbeitsabläufen (Workflows)

Beispiel: Ein Warenkorb kann die Verlinkung auf eine „Bestellungen“-Ressource haben, die bei Create als Argument dann den zu bestellenden Warenkorb als JSON-Objekt erhält und so einen Bestellvorgang initiiert

RESTful Services

CRUD

Die Interaktion eines Benutzers lässt sich mittels einer einfachen Abbildung auf die HTTP-Verben beschreiben (CRUD-Operationen)

Action	Verb
Create	POST
Read (auch Retrieve)	GET
Update	PUT oder PATCH
Delete (auch Destroy)	DELETE

Etwas **überraschend** wird **POST** für **Create** genutzt und PUT für Update. Eigentlich hätte man dies umgekehrt vermutet. Grund ist die **Idempotenz**!

RESTful Services

CRUD

Create

- POST /resourceNames

Read

- GET /resourceNames
- GET /resourceNames/resourceId

Update

- PUT /resourceNames/resourceId
- PATCH /resourceNames/resourceId

Delete

- DELETE /resourceNames/resourceId

RESTful Services

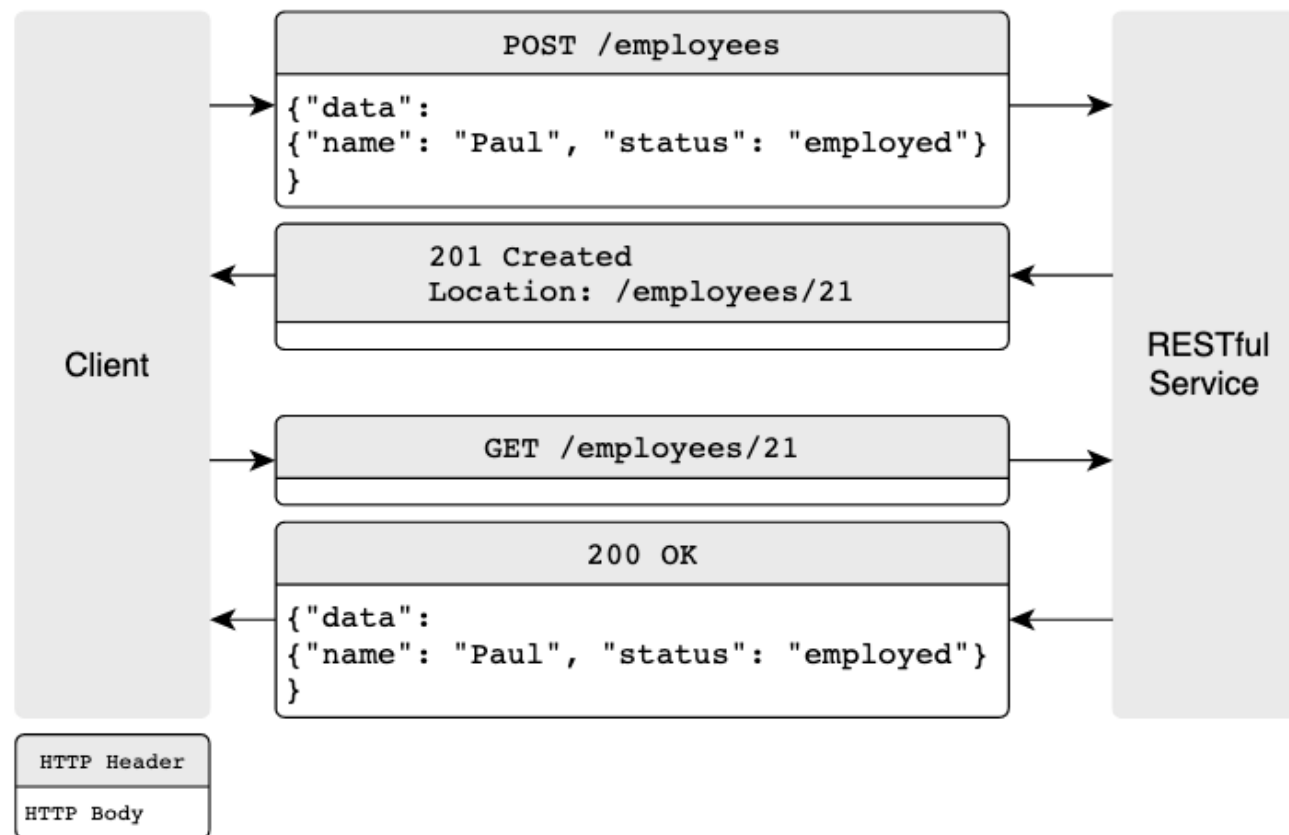
Eigenschaften

- **HTTP als zentrales Protokoll zur Interaktion**
- **Im Mittelpunkt stehen die Ressourcen, die eindeutig adressierbar sind (URL / URI)**
 - > Ressourcen sind alles, was man mit einem Namen bezeichnen kann
 - > Eine Naming Authority weist den Ressourcen eine URI zu (oder der Anwendungsentwickler entscheidet über das Routing)
- **Jede Ressource hat eine oder mehrere Repräsentationen (XML, Text, JSON, PNG, MP3 etc.)**
 - > Repräsentationen stellen den Zustand einer Ressource dar, es handelt sich also um eine flexible Objekt-Serialisierung
 - > Ressourcen besitzen ggf. Verweise auf andere Ressourcen (analog zu Objekt-Referenzen)
 - > Ressourcen werden übertragen, manipuliert und wieder übertragen. Die Repräsentation ist dabei aushandelbar (**Content Negotiation**).
- **"ROA" (Resource-Oriented Architecture)**

- Die genaue Struktur der verwendeten Ressourcen wird in REST nicht fest beschrieben
 - Wir müssen also nicht in explizit festgelegten Objektstrukturen denken, wie sie mittels XML-Schemata definiert werden; eher wie in Script-Sprachen ohne feste Typisierung
 - Das eröffnet Freiheitsgrade, aber auch mögliche Probleme (»Wie genau ist die Struktur der Objekte?«)
 - Prinzipiell kann man auch nur die wichtigen Teile eines Objektes erhalten (PATCH-Operation, Query-Parameter)
- Ressourcen sind oft denormalisiert
 - Man bekommt alles Wichtige mit einer einzelnen GET-Operation
 - Vermeidung externer Objektreferenzen
 - Passt zu der Vorstellung, dass alles, was zum Ausführen der Operation notwendig ist, auch mitgeliefert wird
 - Folgt der JSON-Idee von Embedded Documents
- Testwerkzeuge für REST API: OpenAPI, Swagger, Insomnia

RESTful Services

Die Interaktion



Quelle: <https://phauer.com/2015/restful-api-design-best-practices/>

- REST orientiert sich an **Nomen**. Damit sollte die **URL** auch **nomenorientiert** gebildet werden (**Plural!**)
 - http://api.example.com/device-management/managed-devices http://api.example.com/device-management/managed-devices/{device-id}
 - http://api.example.com/user-management/users/ http://api.example.com/user-management/users/{id}
- Auf Basis von HTTP existieren zwei Möglichkeiten, Anfragen zu parametrisieren; die Art der Parametrisierung muss auf Server-Seite bei der Programmierung berücksichtigt werden
 - **Query-Parameter** können als Argument der URL angehängt werden
 - ?Key1=Value1&Key2=Value2...
 - Es können so komplexe Abfragen gestaltet werden
 - **Path-Parameter** fixieren dynamisch Ressourcen in der URL
 - Server verwaltet die Ressourcen, die unter einer URL verfügbar sind, z.B. users/{id}
 - Create sollte daher die id nicht mitliefern, sondern die Erzeugung dem Server überlassen
 - Ressourcen können auch verschachtelt sein

http://api.example.com/device-management/managed-devices/{device-id}/parameter/{parm-id}

RESTful Services

URLs / Path-Params

- `GET /tickets/12/messages` - Retrieves list of messages for ticket #12
- `GET /tickets/12/messages/5` - Retrieves message #5 for ticket #12
- `POST /tickets/12/messages` - Creates a new message in ticket #12
- `PUT /tickets/12/messages/5` - Updates message #5 for ticket #12
- `PATCH /tickets/12/messages/5` - Partially updates message #5 for ticket #12
- `DELETE /tickets/12/messages/5` - Deletes message #5 for ticket #12

Quelle: <https://www.vinaysahni.com/best-practices-for-a-pragmatic-restful-api#useful-post-responses>

RESTful Services mit Fetch

Content Negotiation

- Der Client überliefert oder fordert eine Ressource in einer speziellen Repräsentation an
- Der Client kann die möglichen bzw. gewünschten Repräsentationen über den HTTP-Header "Accept" angeben
- Der Server gibt die tatsächlich gelieferte Repräsentation durch den HTTP-Header "Content-Type" an
- Bei Node.js Prüfung bzw. Angabe durch `request.headers.get` bzw. `response.headers.get`
- Wenn der Server die angefragte Repräsentation nicht unterstützt, dann wird ein Fehler durch den HTTP-Status 415 ("Unsupported Media Type") geliefert

RESTful Services mit Fetch

Content Negotiation mit dem http-Modul im Browser

```
async function getObj(url, parameter) {  
  let response = await fetch(url, parameter);  
  const contentType = response.headers.get('content-type');  
  if (response.status === 401)  
    throw new Error('Request was not authorized.');
```



```
  if (contentType === null)  
    return new Promise(() => null);  
  else if (contentType.startsWith('application/json;'))  
    return await response.json();  
  else if (contentType.startsWith('text/plain;'))  
    return await response.text();  
  else  
    throw new Error(`Unsupported contenttype: ${contentType}`)  
}
```



```
getObj('https://abc.org/', { headers: { 'Accept': 'text/plain' } })  
  .then(console.log)  
  .catch(console.error)
```


RESTful Services mit Fetch

Content Negotiation

Hinweis: Sobald wir mittels `response.json` auf das erhaltene Objekt zugegriffen haben, ist es zu spät für eine Content Negotiation:

```
// TypeError: body stream already read
fetch(url)
  .then(res => res.json().catch(() => res.text()))
```

Hier hilft ein Fallback-Trick, indem das Response-Objekt geklont wird:

```
fetch(url)
  .then(res => res.clone().json().catch(() => res.text()))
  .then(data => {
    // data is now parsed JSON or raw text
  })
```

RESTful Services Server-Seite

Content Negotiation mit Express

Bei Express bietet sich die **res.format-Methode** an, die in Abhängigkeit vom **Accept-Header** des **Request**-Objekts arbeitet:

```
res.format({  
  'text/plain': function () { res.send('hey') },  
  'text/html': function () { res.send('<p>hey</p>') },  
  'application/json': function () { res.send({ message: 'hey' }) },  
  'default': function () { res.status(415).send('Unsupported') }  
})
```

Verkürzte Notation ohne **vollständigen MIME-Namen** möglich

```
res.format({  
  text: () => res.send('hey'),  
  html: () => res.send('<p>hey</p>'),  
  json: () => res.send({ message: 'hey' })  
})
```

Abfrage des Content-Types auf Client-Seite mittels

```
res.get('Content-Type') // "text/plain"
```

RESTful Services Server-Seite

Content Negotiation mit der Body-Parser-Middleware (alt)

Für Express und POST-Operationen wurde früher das Modul **body-parser** genutzt; die Middleware vereinfacht die Verarbeitung von Body-Parametern (und auch von Query-Parametern); seit 4.16 ist es bereits in Express enthalten und **muss nicht mehr separat eingebunden werden**

```
import express from 'express';
import bodyParser from 'body-parser';
const app = express();

// Verarbeitet application/json für alle Routen (POST, PUT, PATCH)!
app.use(bodyParser.json());

// Klassische Formulardaten application/x-www-form-urlencoded
app.use(bodyParser.urlencoded({ extended: true }));

// Diese POST-Route verarbeitet nun JSON- und URL-Kodierung
app.post('/post', (req, res) => {
  console.log(req.body);
  res.sendStatus(200);
});
```

RESTful Services Server-Seite

Body-Parser

Never trust the user! Was für ein JSON-Objekt bekommen wir?

Note As `req.body` 's shape is based on user-controlled input, all properties and values in this object are untrusted and should be validated before trusting. For example, `req.body.foo.toString()` may fail in multiple ways, for example the `foo` property may not be there or may not be a string, and `toString` may not be a function and instead a string or other user input.

Quelle: <https://www.npmjs.com/package/body-parser>

`bodyParser.json` wird nur den Content-Type `json` übernehmen. Das Middleware-Konzept des **body-parser** erlaubt das Hinzufügen **mehrerer Parser** für weitere Content Types (z.B. `urlencoded`)
Hinweis: Die Verwendung von **response.json** verarbeitet das Response-Objekt endgültig! Spätere Middlewares können dies also nicht noch einmal verarbeiten, `body-parser` löst das Problem

RESTful Services Server-Seite

Ein Server mit Body-Parser und JSON-Verarbeitung

```
import express from 'express';
import bodyParser from 'body-parser';
const app = express();
let todos = [
  { id: 1, title: 'kaufe Milch' },
  { id: 2, title: 'mache Übungsaufgaben' },
  { id: 3, title: 'telefoniere mit Eltern' }
];
app.use(bodyParser.json())
app.get('/todos', (req, res) => res.status(200).json(todos));
app.post('/todos', (req, res) => {
  let newTodo = {}
  newTodo.id = todos.length + 1;
  newTodo.title = req.body.title;
  todos.push(newTodo);
  res.setHeader('Location', '/todos/' + newTodo.id);
  res.status(201).json();
});
```

RESTful Services Server-Seite

Ersatz für Body-Parser

Seit Express 4.16 gibt es eine eigene Middleware in Express, die die Body-parser Funktionalität übernimmt:

Alt:

```
app.use(bodyParser.json());
```

Content-Type:
application/json

Neu:

```
app.use(express.json());
```

Alt:

```
app.use(bodyParser.text());
```

Content-Type:
text/plain

Neu:

```
app.use(express.text());
```

Alt:

```
app.use(bodyParser.urlencoded({ extended: true }));
```

Content-Type:
application/x-www-
form-urlencoded

Neu:

```
app.use(express.urlencoded());
```

RESTful Services Server-Seite

Verarbeitung der Requests

```
// Middleware für die Verarbeitung von Text
app.use(express.text());

app.post('/action', function(req, res) {
  let name = req.body;
  // Weitere Verarbeitung
});
```

RESTful Services Server-Seite

Verarbeitung der Requests

Beispiel URL-kodierte Daten (Formulardaten): name=foo&color=red

Beispiel JSON-kodierte Daten: {"name": "foo", "color": "red"}

```
// Einheitliche Middlewares für die Verarbeitung von  
// JSON- oder URL-kodierte Daten
```

```
app.use(express.json());
```

```
app.use(express.urlencoded());
```

```
app.post('/action', function (req, res) {
```

```
  let name = req.body.name;
```

```
  let color = req.body.color;
```

```
  // Weitere Verarbeitung
```

```
});
```


RESTful Services Server-Seite

Never trust the User?

Problem: Wir können die Struktur des empfangenen JSON-Objektes nicht direkt mit `instanceof` prüfen, weil es keine Prototyp-Verbindungen gibt.

JSON ist immer vom Typ Object.

Mittels des bereits behandelten `express-validator` müssen daher alle Felder einzeln getestet werden.

Komfortabler sind die Ansätze, mittels eines "JSON-Schemas" und der **`express-json-validator-middleware`**

RESTful Services Server-Seite

Bisher: Express-Validation

```
const express = require('express')
const { validate, ValidationError, Joi } = require('express-validation')

const loginValidation = {
  body: Joi.object( schema: {
    email: Joi.string()
      .email()
      .required(),
    password: Joi.string()
      .regex( pattern: /[a-zA-Z0-9]{3,30}/)
      .required(),
  })
})

const app = express();
app.use(express.json())

app.post( path: '/login', validate(loginValidation, options: {}, joiRoot: {}), (req :... , res :... ) => {
  res.json( body: 200)
})

app.use(function(err, req, res, next) {
  if (err instanceof ValidationError) {
    return res.status(err.statusCode).json(err)
  }

  return res.status(500).json(err)
})

app.listen( port: 3000)
```

RESTful Services Server-Seite

Neu: Validierung über ein Schema

```
// Define a JSON Schema
var StreetSchema = {
  type: 'object',
  required: ['number', 'name', 'type'],
  properties: {
    number: {
      type: 'number'
    },
    name: {
      type: 'string'
    },
    type: {
      type: 'string',
      enum: ['Street', 'Avenue', 'Boulevard']
    }
  }
}
```

RESTful Services Server-Seite

Neu: Validierung über ein Schema

```
var express = require('express');
var bodyParser = require('body-parser');

var { Validator, ValidationError } = require('express-json-validator-middleware');

// Initialize a Validator instance first
var validator = new Validator({allErrors: true}); // pass in options to the Ajv instance

// Define a shortcut function
var validate = validator.validate;

var app = express();

app.use(bodyParser.json());

// This route validates req.body against the StreetSchema
app.post('/street/', validate({body: StreetSchema}), function(req, res) {
  // At this point req.body has been validated and you can
  // begin to execute your application code
  res.send('valid');
});

// Error handler for validation errors
app.use(function(err, req, res, next) {
  if (err instanceof ValidationError) {
    // At this point you can execute your error handling code
    res.status(400).send('invalid');
    next();
  }
  else next(err); // pass error on if not a validation error
});
```

RESTful Services Server-Seite

Path-Parameter

```
app.use(express.json());
```

```
app.put('/cars/:id', (req, res) => {  
  let id = parseInt(req.params.id);  
  if (cars[id]) { // existiert das Auto?  
    let updatedCar = req.body;  
    cars[id] = updatedCar;  
    res.status(200).send();  
  } else {  
    res.status(404, 'Car not found').send();  
  }  
});
```

Never Trust the User:
id muss kein Int sein

```
app.get('/cars:cardId/passengers/:passengerId', (req, res) => {  
  // Access cardId via: req.params.cardId  
  // Access passengerId via: req.params.passengerId  
})
```

RESTful Services Server-Seite

Content Negotiation

Prüfung, in welcher Repräsentationsform die Ressource gewünscht wird: Zugriff auf Accept-Header über `res.format()`

```
app.get('/', (req, res) => res.format({  
  json: () => res.status(200).json(cars),  
  html: () => res.status(406).send('Not supported')  
}));
```

Beachte: bei der Verarbeitung von Query-Parametern für GET benötigt man den body-parser nicht:

```
app.get('/', (req, res) => {  
  let id = req.query.id;  
  res.status(200).json(cars[id])  
});
```

RESTful Services

Wie sollten Query-Parameter genutzt werden?

Filtering: Use a unique query parameter for each field that implements filtering. For example, when requesting a list of tickets from the `/tickets` endpoint, you may want to limit these to only those in the open state. This could be accomplished with a request like `GET /tickets?state=open`. Here, `state` is a query parameter that implements a filter.

Sorting: Similar to filtering, a generic parameter `sort` can be used to describe sorting rules. Accommodate complex sorting requirements by letting the sort parameter take in list of comma separated fields, each with a possible unary negative to imply descending sort order. Let's look at some examples:

- `GET /tickets?sort=-priority` - Retrieves a list of tickets in descending order of priority
- `GET /tickets?sort=-priority,created_at` - Retrieves a list of tickets in descending order of priority. Within a specific priority, older tickets are ordered first

Quelle: <https://www.vinaysahni.com/best-practices-for-a-pragmatic-restful-api>

RESTful Services

Wie sollten Query-Parameter genutzt werden?

Searching: Sometimes basic filters aren't enough and you need the power of full text search. Perhaps you're already using [ElasticSearch](#) or another [Lucene](#) based search technology. When full text search is used as a mechanism of retrieving resource instances for a specific type of resource, it can be exposed on the API as a query parameter on the resource's endpoint. Let's say `q`. Search queries should be passed straight to the search engine and API output should be in the same format as a normal list result.

Combining these together, we can build queries like:

- `GET /tickets?sort=-updated_at` - Retrieve recently updated tickets
- `GET /tickets?state=closed&sort=-updated_at` - Retrieve recently closed tickets
- `GET /tickets?q=return&state=open&sort=-priority,created_at` - Retrieve the highest priority open tickets mentioning the word 'return'

Quelle: <https://www.vinaysahni.com/best-practices-for-a-pragmatic-restful-api>

RESTful Services

Wie sollten Query-Parameter genutzt werden?

Limiting which fields are returned by the API

The API consumer doesn't always need the full representation of a resource. The ability select and chose returned fields goes a long way in letting the API consumer minimize network traffic and speed up their own usage of the API.

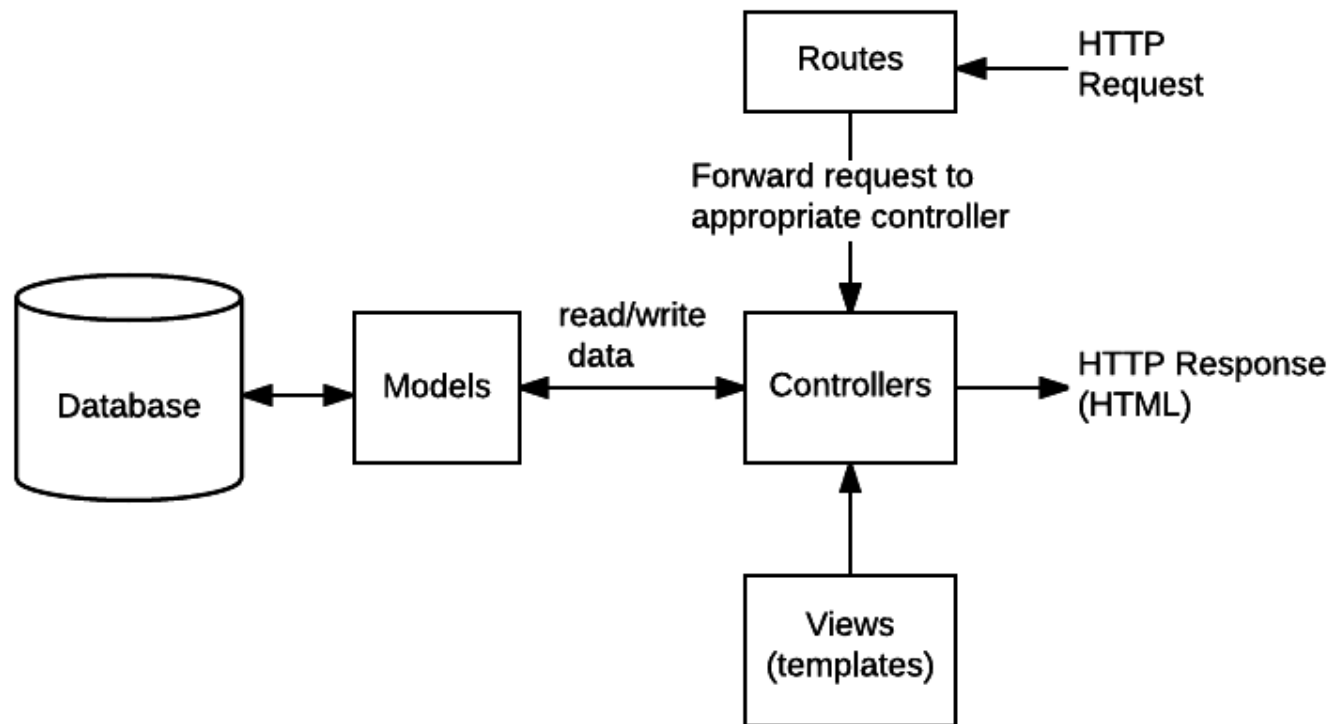
Use a `fields` query parameter that takes a comma separated list of fields to include. For example, the following request would retrieve just enough information to display a sorted listing of open tickets:

```
GET /tickets?fields=id,subject,customer_name,updated_at&state=open&sort=-updated_at
```

Quelle: <https://www.vinaysahni.com/best-practices-for-a-pragmatic-restful-api>

RESTful Services Server-Seite

Überblick



RESTful Services Server-Seite

Erster Ansatz: Routes erstellen

Benutzerdefiniertes Wiki-Modul:

```
// wikiRoutes.js
import express from 'express';
const router = express.Router();

router.get('/', (req, res) => res.send('Wiki Homepage'))
router.get('/about', (req, res) => res.send('About Wiki'))
router.post('/about', (req, res) => res.send('About Wiki'))
```

```
export default router;
```

```
// app.js
import express from 'express';
import router from './wikiRoutes.js';
const app = express();
app.use('/wiki', router);
```

RESTful Services Server-Seite

Besserer Ansatz: Express

```
// taskApp.js
import express from 'express';
import router from './taskRoutes.js';
const port = 3001;
const app = express();

app.use(express.text());
app.use('/tasks', router);

app.listen(port, () => console.log("Listening on http://localhost:" + port));
```

RESTful Services Server-Seite

Besserer Ansatz: Routen

```
// taskRoutes.mjs
import express from 'express';
import { check, validationResult } from 'express-validator';
import * as controller from './taskController.mjs';
const router = express.Router();

router.route('/')
  .get(controller.readAllTasks)
  .post(controller.createTask)
  .delete(controller.deleteAllTasks);

router.route('/:id([0-9]+)')
  .get(controller.readTask)
  .delete(controller.deleteTask)
  .patch(controller.setDone)

export default router;

function validationError(req, res, next) {
  const errors = validationResult(req);
  if (!errors.isEmpty()) {
    console.log(errors)
    return res.sendStatus(422);
  } else {
    next();
  }
}

router.use('/:id', check('id').isNumeric(),
  validationError);
```

RESTful Services Server-Seite

Besserer Ansatz: Controller

```
// taskController.js
import taskList from './taskList.js';
const tasks = new taskList();
export function createTask(req, res) {
  let title = req.body;
  if (!title) return res.sendStatus(400);
  let id = tasks.create(title);
  res.setHeader('Location', req.originalUrl + '/' + id);
  res.status(201).json(tasks.read(id));
}
export function readTask(req, res) {
  let task = tasks.read(parseInt(req.params.id));
  if (!task) return res.sendStatus(404);
  res.json(task);
}
export function deleteTask(req, res) {
  if (tasks.delete(parseInt(req.params.id))) { res.sendStatus(200); }
  else { res.sendStatus(404); }
}
...
```

RESTful Services Server-Seite

Besserer Ansatz: Model

```
// taskList.js
export default class {
  #data = new Map();
  #counter = 0;

  readAll() { return Array.from(this.#data.values()); }

  read(id) { return this.#data.get(id); }

  create(title) {
    let task = { id: this.#counter, title: title, done: false }
    this.#data.set(this.#counter, task);
    this.#counter++;
    return task.id;
  }
  ...
}
```

RESTful Services Client-Seite

Verarbeitung

Wir können hier beispielsweise die FETCH-API nutzen!

```
// Create a new task
fetch('https://localhost:3001/tasks', {
  headers: { 'Content-Type': 'application/json; charset=utf-8' },
  method: 'POST',
  body: 'Abendessen kochen'
}).then(
  // status code should be 201 & body should be new task
).catch(
  // error handling
);
```


- Wer liefert beim Create die ID?
- IDs sollten nur beim PUT vom Client mitgeben werden, denn ein PUT ist normalerweise ein Update, so dass die ID schon bekannt ist
- Beim POST sollte keine ID mitgegeben werden bzw. sie sollte vom Server ignoriert werden
- Location Header bei der 201-Response sollte die URL mit der neuen ID zurückliefern!

Updates & creation should return a resource representation

A PUT, POST or PATCH call may make modifications to fields of the underlying resource that weren't part of the provided parameters (for example: created_at or updated_at timestamps). To prevent an API consumer from having to hit the API again for an updated representation, have the API return the updated (or created) representation as part of the response.

In case of a POST that resulted in a creation, use a [HTTP 201 status code](#) and include a [Location header](#) that points to the URL of the new resource.

Quelle: <https://www.vinaysahni.com/best-practices-for-a-pragmatic-restful-api#useful-post-responses>